

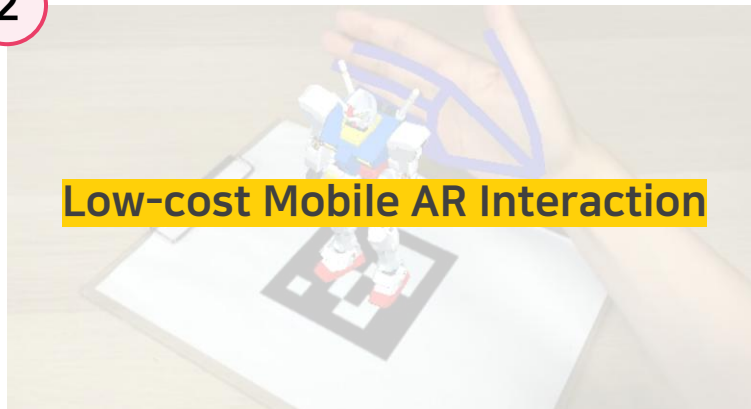
1



Shrunken Reality

현실세계에서 일어나는 이벤트를 현실을 **축소 시킨 미니어처**의
동일한 위치에 실시간으로 증강하는 **확장된** 개념의 **증강현실**

2



Low-cost Mobile AR Interaction

[ETRI 산학 협업] **단안 스마트폰 카메라** 한 대만을 이용하여
손과 가상 오브젝트 사이의 **Occlusion** 및 **Collision**
처리 가능한 저비용 모바일 증강현실 상호작용 시스템

3



J.A.R.V.I.S

영화 '**아이언맨2**'의 **홀로그램 장면**을 재현한 프로젝트
실제 공간을 **디지털 트윈 공간**으로 생성 및 가공하여
사용자의 **제스처를 인식**하는 시스템

축소 현실 (Shrunkten Reality), 확장된 개념의 증강현실을 만들어보자!



배경

다중 카메라 및 기존 AR 시스템의 한계점: 여러 대의 CCTV로 전체 공간을 이해하기 어렵고, 기존 AR은 사용자가 직접 해당 공간에 있어야 하는 공간적 제약과 대형 공간에서 정보를 한눈에 파악하기 어렵다는 문제.

→ 따라서 큰 공간의 상황을 한눈에 축소시켜 직관적으로 관찰하고 이해할 수 있는 새로운 시스템이 필요.

담당 파트

현실 및 축소 세계 매핑: 웹캠과 노트북을 사용해 실제 건물 및 3D 프린터로 출력한 미니어처 건물을 매핑.

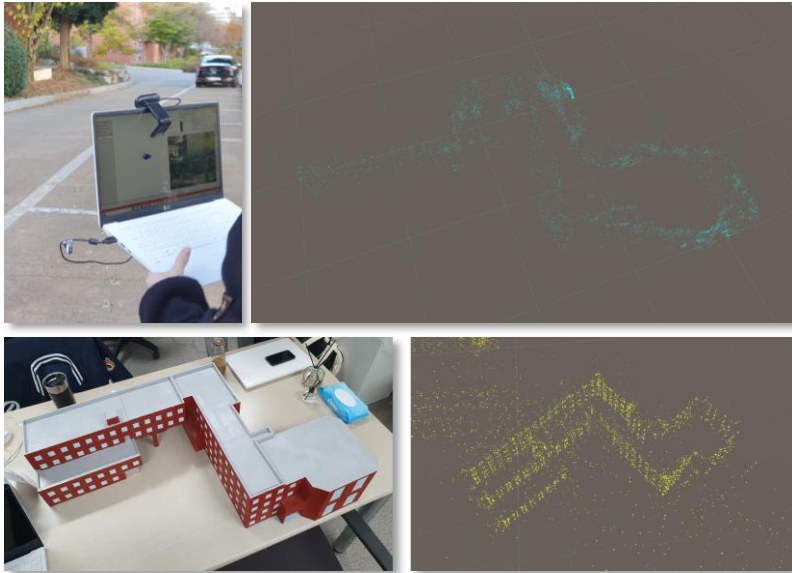
캘리브레이션 및 실시간 증강: 카메라 캘리브레이션을 사전에 진행. 저장된 맵을 기반으로 re-localization을 수행.

→ 'CCTV(2D) → 현실(3D) → 축소세계(3D) → 사용자 시점(2D)'의 변환을 실시간으로 계산.

실시간 이벤트(위치) 포착: ORB-SLAM2와의 연동을 위해 YOLO(C++)를 도입.

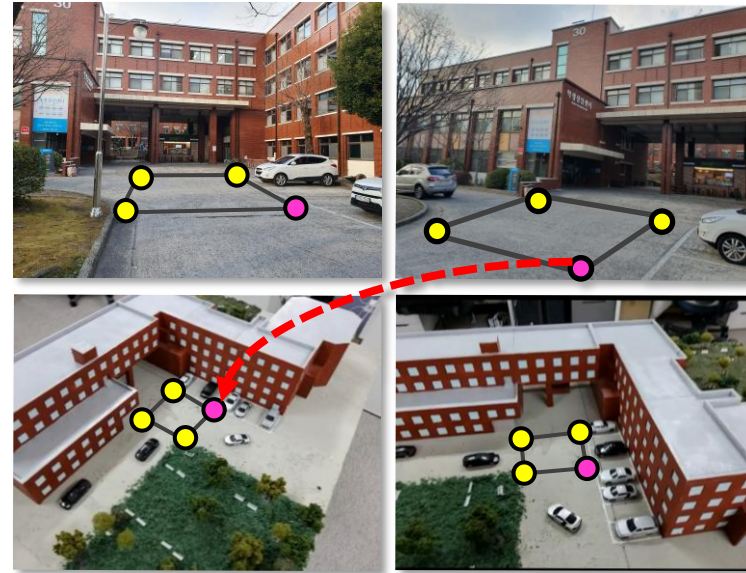
효율적인 연산을 위해 포착 대상을 사람의 발 위치로 한정하여 추적 및 매핑.

👤 주요 성과 - 1



현실 세계 & 축소 세계 매핑 결과

👤 주요성과 - 2



현실세계-축소세계 변환 관계 설정

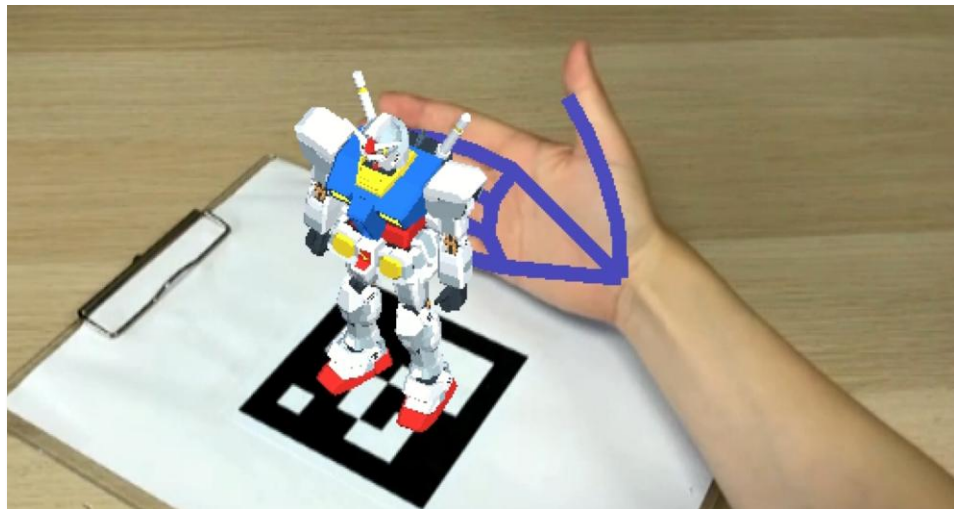
C++ 기반의 YOLO 통합 (어려웠던 점)

OpenCV 버전 제약: ORB-SLAM2가 **구버전 OpenCV(3.x 초반)**를 기반으로 동작 → 최신 딥러닝 모듈 적용을 위해 **4.x로 업그레이드하기엔 시스템 전체를 교체**해야 하는 큰 부담.

구버전 YOLOv3 도입 배경: OpenCV 3.x에서 **실험적 dnn 모듈**을 쓰기 위해 opencv_contrib 확장을 이용 → GPU 호환성으로 인해 구버전 YOLOv3를 채택함.

대안책: 라이브러리를 직접 통합하는 대신, 내부망을 통해 **Python과 C++ 간 통신 기반**으로 동작하게 만드는 방식도 대안이 될 수 있음.

스마트폰 카메라만으로 증강현실 인터랙션을 해보자!



배경

스마트폰 기반 AR 시스템의 한계점: 별도의 Depth 센서나 외부 트래커가 없는 스마트폰 기반 저비용 MR 환경 (Google Cardboard 등)은 사용자가 가상 오브젝트와 상호작용할 수단이 거의 없음. 화면 속 가상 객체는 떠 있고 손은 그 옆을 무관하게 지나가는 상황.

→ 일반 스마트폰 카메라 한 대만 가지고도 손과 가상 객체가 같은 공간에 존재하게 할 수 있는 시스템 필요.

담당 파트

Python 기반 공간 추적 백엔드 구현: 팀원은 Unity 기반 시각화 및 손-오브젝트 간 충돌 인터랙션 구현 등 프론트엔드 담당.

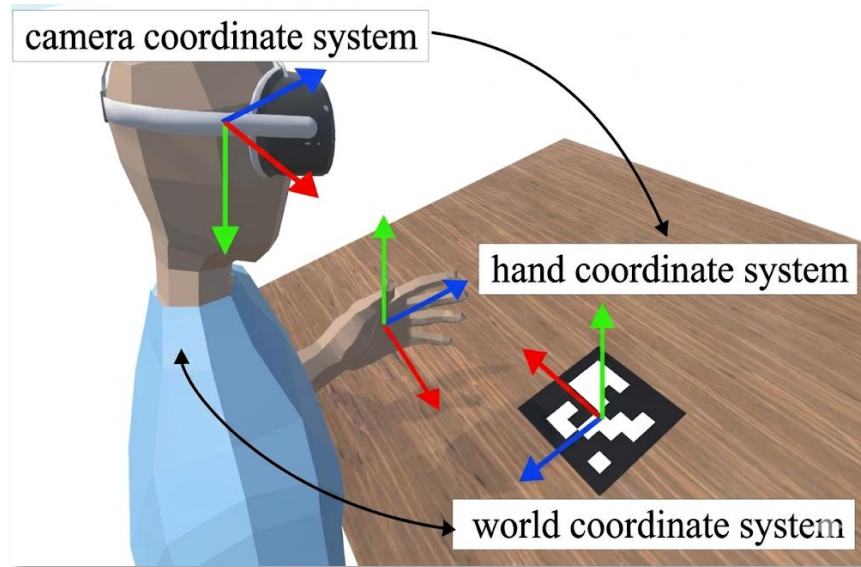
월드/카메라/손 좌표계 설정: ArUco 마커를 이용하여 월드를 정의. PnP 알고리즘으로 월드-카메라 간 포즈 행렬 계산.

MediaPipe와 PnP 알고리즘으로 손-카메라 간 포즈 행렬 계산.

손 21개 관절의 월드 정합: 카메라 좌표계(C)를 매개체로 하여 손가락 21개 관절 랜드마크 위치를 월드 좌표계로 표현할 수 있게 됨.

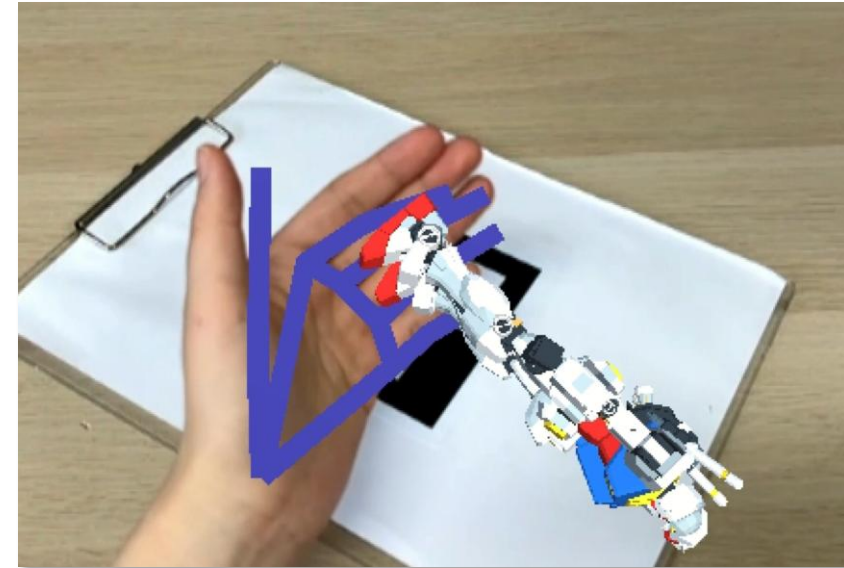
$$P_W^i = T_{WC} T_{CH}^i \quad \text{for } i \in [1 \dots 21]$$

👤 주요성과 - 1



월드/카메라/손 좌표계 시각화

👤 주요성과 - 2



어려웠던 점

MediaPipe 좌표계 문제점: MediaPipe가 자체적으로 좌표계를 반환하는 손의 3D 좌표는 **카메라 중심의** 미터(m) 단위라 **월드 좌표계와 호환 불가**.

→ 중지(가운데 손가락)의 손바닥과 연결된 관절을 Hand 좌표계의 원점으로 설정하여 **카메라를 매개로 두 단계 변환을 합성**하여 해결.

궁극적 한계: 소프트웨어 방식의 인터랙션이므로 손 자체에 제공되는 **물리적 피드백이 없음**.

디지털 트윈과의 인터랙션을 구현해보자!



배경

영화 '아이언맨 2'의 토니 스타크는 빈 공간에 손을 움직이는 것만으로 3D 홀로그램을 자유자재로 조작. 그러나 현실에서 3D 포인트클라우드 데이터는 보통 키보드와 마우스로 다룸.

- 3차원 데이터를 2차원 입력장치로 제어한다는 아쉬움과 부조화가 존재.
- 실제 공간을 그대로 디지털로 옮기고, 그 위에서 사람의 손으로 직접 조작하는 인터페이스가 필요.

담당 파트

팀원: iPad LiDAR 스캐너로 공간 스캐닝, 제스처 (회전, 확대, 이동, 선택 등) 인식

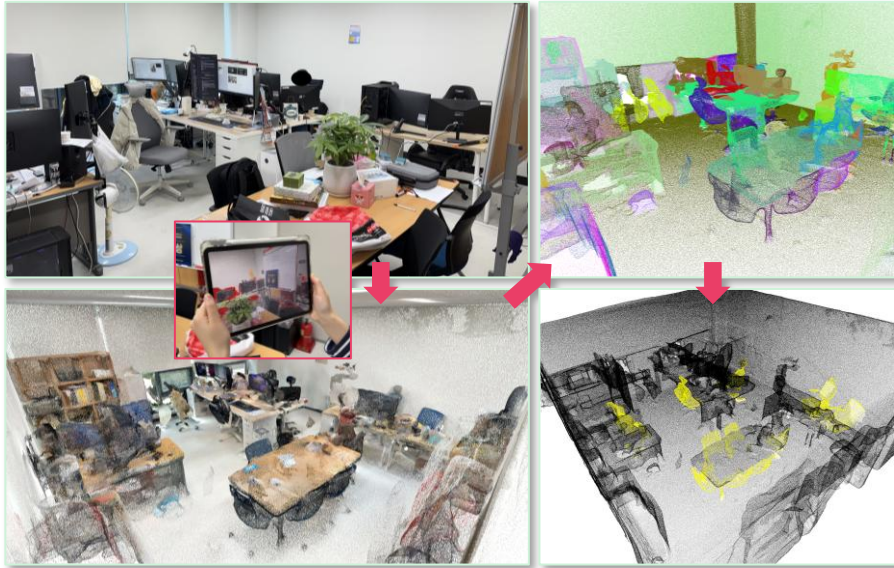
본인: 포인트클라우드의 객체 단위 분리, RGB-D 카메라 기반 손동작 인터랙션 구현. (선택한 물체 인식)

공간을 조작 가능한 물체들로 변환: PointTransformer를 이용하여 raw 데이터에 **Panoptic Segmentation** 수행 → 개별적으로 조작 가능한 물체 단위로 클러스터링.

현재 선택하고자 하는 물체 인식: 모든 인스턴스들의 위치(xyz)와 양 손의 위치($2 * xyz$)를 실시간으로 거리 계산.

→ 유의미한 거리 내로 들어오는지 확인 → **유저가 다루고자 하는 물체**를 1차적으로 선정할 수 있는 알고리즘 구현.

👤 주요성과 - 1



실제 공간을 세그멘테이션한 모습

👤 주요성과 - 2



인스턴스의 중심들 (검은색 구)과 왼손 (빨간색 구),
오른손 (파란색 구), 손과 손 사이의 거리 (초록색 선분)를 시각화

어려웠던 점

데이터 분포 문제: PointTransformer가 표현할 수 있는 클래스 종류가 연구실 공간에 있는 **모든 물체의 클래스를 표현할 순 없음.**

LiDAR 스캐닝 문제: 창문처럼 **유리가 있는 부분은 LiDAR 빛이 투과되어 창문 밖 공간이 왜곡 표현되는 문제 발견.**

→ 복잡한 해결책 보단, 실내 공간에서의 인터랙션임을 생각하여 벽과 천장 내부로 range를 한정하며 편리하게 해결.

궁극적 한계: 영화 속 장면처럼 증강을 위한 빛이 공중에서 멈추는 하드웨어적 방식이 불가. → 원격 수술처럼 디스플레이를 보며 조작하는 방식을 채택.